Agent 时代，
学生如何做出
有价值的开源贡献?

Agent是放大器：怎样补足短板，逐步形成自己的判断

更容易动手，不等于更容易被需要；借工具开始，在真实问题中成长。

今年的 Issue / PR，已经超过此前累计

kubeedge / ianvs	此前累计	2026年至今
Issue	87	266
PR	211	346
合计	**298**	**612**

我的观察：Agent是增量的重要推动因素，审查时间却没有同比增加。

我们已引入AI辅助review；更依赖贡献者的文档、验证和例会讲解。

Agent改变的不只是速度，也可能改变分工：我们还需要贡献什么？

GitHub新建数量（全部状态，不等于AI生成比例）；截至2026-11-03 16:17

开源之夏2026：模式、报酬与任务定位都在变

2025：基础 / 进阶

¥8,000 / ¥12,000

按项目任务申请，完成并通过结项。

2026：经典 + 领航

经典：¥3,500 / ¥5,000

领航：¥6,000 / ¥8,000 / ¥10,000

变化不只是“12,000 → 5,000”

全年开放；更聚焦基础软件与RISC-V；领航模式由学生挖掘问题、提出方案。

“有些旧课题的编码，今天可能只需约100元token。”非官方成本统计。

我的体会：实现成本降低后，更值得投入的是问题选择、方案判断与完整交付。

来源：2025官方规则、2026学生指南与启动公告；金额均为税前结项报酬

Agent既补足执行短板，也放大工程判断

执行经验不足时

更快把想法做出来

解释代码，补齐API与局部实现
辅助配环境、定位错误与测试
常规编码任务的差距可能缩小

已有工程判断时

更大范围地把事做成

确定方向、拆任务、组织多Agent
设计验收、检查依赖并及时纠偏
经验可转化为更强的交付能力

实现结果接近 ≠ 综合能力相同；所有水平的贡献者都可以使用Agent。

同一个人的成长路径：先借Agent补短板，再在真实任务中积累判断力。

动手门槛降低，不等于贡献机会自动增加

个人端：能做得更多

过去卡住新人的一部分工作，
今天可以借Agent更快推进。

更容易完成局部实现、做出demo；
可以更早参加真实项目，边做边学。

社区端：未必更缺人写代码

资深工程师 + 多个Agent，
可承担部分原需多人实现的工作。

只按明确要求补一段常规实现，
不一定还是社区最稀缺的帮助。

没有变：真实需求、工程质量、贡献责任与持续协作。

常规任务可能减少，新项目也可能增加；不等于新手没有机会，或小贡献失去价值。

学生策略：借工具尽早进入真实问题，持续增加理解、证据与后续投入。

讲者观察与机制分析；不代表所有社区、所有任务均发生同样变化。

变的是参与方式，不变的是贡献的价值

Agent带来的变化	没有变的要求	学生可以怎样做
阅读、实现与调试有了助手	仍需建立自己的理解	不必全学会才开始 在运行和修改中补基础
代码与demo更容易产出	仍要解决真实问题	先调查需求与已有实现 再决定做什么、为何做
成品外观更难说明能力	仍需对后续工作负责	展示复现、取舍与修改 而不只展示漂亮成品
提交增加，AI也参与审查	仍需质量、共识与信任	补验证、写文档、帮review 让已有工作更容易推进

更早开始，在真实任务中学习；不只生成一个实现，还能把工作继续推进。

同一个社区，我先后站在了三种位置

2023

学生贡献者

LFX pretest第一名，入选项目
同年参加开源之夏
做评测、数据与云边协同

2024 起

项目导师

设置任务与pretest
选择学生、讨论方案
协助把握项目进度

现在

Reviewer

先判断修改有没有必要
再确认旧示例的兼容性
之后才深入review

2023—2024年：我主要靠手工配环境、读报错和debug，没有用上今天这样的coding agent。

不必把当年的机械摸索重走一遍；但对问题、代码和结果的理解，仍要自己建立。

实现更容易之后，更要选你真正关心的问题

01 我的研究问题

硕士研究：边缘智能、终身学习。

02 社区里的相关工作

KubeEdge / Ianvs的方向与研究接近。

03 开源反过来支持研究

相关工作形成LAECIPS论文。

论文成果


Journal of Industrial Information Integration 48 (2025) 100955

Contents lists available at ScienceDirect

Journal of Industrial Information Integration

journal homepage: www.elsevier.com/locate/jii

ELSEVIER

Full Length Article

LAECIPS: Large vision model assisted adaptive edge–cloud collaboration for IoT-based embodied intelligence system☆

Shijing Hu [a], Zhihui Lu [a,*], Xin Xu [b,*], Ruijun Deng [a], Xin Du [c], Qiang Duan [d]

Journal of Industrial Information Integration

2025年发表 · 中科院一区 TOP

场景经验也有价值：从你真正遇到的问题出发，补充社区缺少的信息与验证。

有了 Agent，为什么仍值得跟着社区导师做项目？

申请前，我已旁听三个月社区例会

先接触社区，后来再申请。

三个月是我的经历，
不是学生必须完成的准备期。

1 任务目标相对明确

在真实场景里确认：谁需要、做什么、怎样验收。

2 有导师和阶段节奏

工具可以给建议；导师帮助核对方向、取舍和阻塞。

3 可以深度参与一个项目

有结项支持与奖励；也要持续投入和反馈。

工具能补实现；导师与社区反馈帮助你积累需求判断、方案取舍和协作经验。

CNCF LFX Mentorship：跟着社区导师做项目

怎样组织?

Linux Foundation：LFX平台
CNCF：组织旗下社区参与
社区导师：出题、选人、指导
Mentee：在项目周期内贡献

按期次申请，有期限与任务要求。
津贴及资格按具体项目规则。

怎么报名?

1 登录LFX，创建Mentee资料

2 筛选 Accepting Applications 项目

3 读要求与期次，点击 Apply

4 补齐材料与任务，按项目要求做pretest

入口：mentorship.lfx.linuxfoundation.org

Linux Foundation官方申请指南；CNCF参与的项目使用LFX平台

开源之夏：选社区课题，或从方向中提出方案

怎样组织?

中国科学院软件研究所发起
开源组织 / 企业发布任务或方向
社区导师指导，组委会审核
学生贡献代码并完成结项

2026全年开放、按项目周期推进。
同一时间最多申请 / 承担一个项目。

经典模式的报名路径

1. 官网学生入口注册，提交学籍等资料
2. 从项目列表选题，按公示邮箱联系导师
3. 撰写申请书，在系统提交
4. 导师与组委会审核；中选后签约开发

领航模式：先选方向、投提案、参与社区沟通，再提交正式方案。

入口：summer.ospp.ac.cn → 学生登录 / 项目列表

2026学生指南；面向年满18周岁且项目周期内在读的高校学生

先弄清选拔方式：两条路线，准备重点不同

Ianvs 的 LFX 课题

导师设pretest，按完成质量评分；
我们相关课题由最高分学生入选。

沟通重点：理解题目与评分要求。
准备重点：按要求完成并可复现。

本社区规则示例：#130

开源之夏：经典模式申请

申请书质量是最主要标准；
前期沟通帮助理解需求、形成方案。

沟通重点：方向、范围、验收方式。
准备重点：具体可信的申请方案。

2026官方导师指南

范围说明：并非所有CNCF项目统一设置pretest；各项目与模式的规则需单独看。

先按规则分配精力：沟通不能代替pretest成绩，漂亮文字也不能代替可行方案。

CNCF统一选拔指南；开源之夏2026导师指南分别见页内链接

LFX 实例：我出的 pretest，到底在考什么？

目标：在 Ianvs 中搭建 LLM 评测，并尝试云边协同算法

50	**评测示例**	选择数据集，建立能运行的LLM benchmark。
25	**云边协同算法**	基于评测示例实现算法，例如联合推理或投机解码。
25	**报告与复现文档**	提交算法设计、实验结果和别人能照做的README。

原题规则：无法按代码与README直接运行，第1、2项得零分；评分也考虑代码质量。

先确保评分项的核心产出能复现，再做题目以外的扩展。

借Agent补编码短板，用真实任务练判断

以 #130 pretest 为例：沿一条运行路径，边做边建立理解

1　让Agent解释，再回到代码核对

找到启动入口、配置与数据流；
把解释对应到实际文件和调用位置。

2　先跑通最小链路，再按要求补齐

把50/25/25分逐项对应到产出；
保留环境、命令和输出，从README重跑。

3　预测关键行为，再运行验证

改一个参数前，先判断它会影响什么；
结果不符时，继续定位而非照抄解释。

每次多积累一个判断

为什么值得做？
关键依赖在哪里？
什么结果才算对？
Agent哪里可能错？
下次能独立判断什么？

不必等成为senior再参与；先从一个小范围开始，逐步学会判断、验证与纠偏。

第一次联系导师：有想法就可以聊，不必等做完

我希望先了解什么？

你为什么对这个项目感兴趣？
你怎样理解要解决的问题？
已有经验与它哪里相关？
时间和设备是否支持？

先判断方向和合作是否契合。

一次交流结束，最好确认这些问题

范围	主要做算法、工程接入，还是评测？
验收	怎样的结果才算完成？什么暂时不做？
条件	现有代码、数据和设备从哪里获得？
下一步	下一轮带什么材料或进展回来讨论？

Agent帮你整理问题，导师帮你确认项目语境；第一次可以先聊，不必等做完。

第二次开始拿成果，再把交流写进申请书

上次确认的问题 → **这次实际推进的内容** → **申请书里的依据**

需求理解	谁遇到什么问题；为什么已有功能不够。
实现方案	相关模块、方案取舍；已有成果能说明什么。
阶段与验收	每阶段交付什么，用哪些结果判断完成。
资源与风险	设备、数据、时间限制；受阻时怎样调整。

申请书不只是写得流畅：让需求、方案与计划，都有真实交流和实际探索作依据。

开源之夏：申请书质量是最主要标准；须经导师与组委会审核

demo仍然重要，还要看能否继续推进

我的选人经验

基本达到项目
最低结项标准

不是只点亮一个功能，
也不必每个细节都已完善。
这能体现投入与完成能力；
达到者少，在我的经验中竞争力强。

今天，展示成果时再讲清

复现：结果怎样得到？

环境、命令、输入与输出能否对应。

取舍：为什么选这个方案？

试过什么、为何放弃、还有什么限制。

修改：反馈之后怎样推进？

收到建议，能继续定位、调整和验证。

首次联系仍可先聊想法；开源之夏正式项目不得提前开发，demo经验不是报名要求。

Agent帮你把demo做得更完整；复现、取舍与后续修改，让理解可见。

让社区认识你：持续参与，比反复表达兴趣更有用

01 带具体问题参加例会

先旁听；熟悉后带一个问题或进展，不必每次都发言。

02 做确有帮助的小贡献

可复现的Issue、准确的文档修正、范围清楚的修复。

03 收到建议之后继续推进

回写结果、更新修改，让别人看到你的合作方式。

没有入选怎么办?

名额有限，不等于
没有贡献资格。

可以继续讨论和提交；
但不默认享有固定导师、
津贴或排期承诺。

漂亮的自我介绍容易生成；持续回应、如实说明、推进问题，才能让社区认识你。

有demo之后，正式项目阶段还有什么工作?

先与导师确认

现有成果覆盖了哪些要求?
哪些需要扩展或接入?
怎么拆阶段与PR?
用什么办法验收?

不要默认“demo能跑”
就等于全部社区工作结束。

Agent参与开发的具体循环

1 **先确认范围**

讨论设计、接口与不改动的部分。

2 **分阶段实现**

每次形成可审查的改动；保留运行记录。

3 **验证与反馈**

新目标、旧示例一起检查；受阻及时沟通。

把省下的机械劳动投入验证、适配和协作；完整贡献不等于一次塞进更大的PR。

我深入 review 之前，先看这两件事

01

修改有没有必要?

修复了什么bug?
新特性解决谁的需求?
为什么现有能力还不够?

02

旧examples还兼容吗?

改动涉及什么公共部分?
哪些旧示例可能受影响?
是否有回归验证与未测说明?

两点都满足，再深入看设计与代码实现。

实现更容易，不代表每项功能都值得合入；社区还要使用和维护它。

Motivation怎么写：说清问题，而不是宣称“优化”

如果是修bug

发生在什么场景?
预期与实际行为有什么差异?
哪个最小步骤可以复现?
为什么需要这次改动?

如果是加特性

谁需要、在哪种场景使用?
现有能力为什么不能满足?
是否已有类似实现或计划?
改动范围先与社区确认了吗?

让Agent先做调查：查文档与现有实现、已有Issue/PR，再复现或比较方案。

先确认为什么值得做，再让Agent帮你做；生成一个建议不等于发现了真实需求。

为什么新功能跑通了，我还要问旧示例?

旧示例 A

旧示例 B

新示例 C

↓ ↓ ↓

公共部分：数据接口 / 执行流程 / 依赖环境

本次目标
让新示例C完成预期任务

还要确认
这次公共改动有没有改变A、B的行为

Ianvs有不同示例与学习范式；改公共代码时，不能只看自己新增的那个。

让 Agent 帮你做回归，不要只让它说“没影响”

先找影响范围，再在可比条件下运行相关示例，记录证据。

检查对象	修改前	修改后	交付证据
原bug / 新目标	记录原现象 或已有能力	验证预期行为 与验收条件	输入、命令、输出 前后差异
相关旧examples	保存基线状态 已有失败单列	相同输入与环境 检查新增失败	逐项结果与日志 不要只写“全通过”
无法运行的项	明确缺数据、 设备或其他条件	标明未验证 约定补测方式	剩余风险与 需要的社区帮助

静态分析给线索；实际运行给证据。没测的如实写，旧失败与新回归分开。

个人写得更快，社区还需要一起把贡献推进下去

我们的应对：AI辅助审查 + 贡献者补上下文 + 维护者作取舍

AI辅助review

梳理改动与潜在问题
帮助检查代码一致性
提供线索，可能误判

贡献者负责讲清

为什么做、方案怎么选
新旧示例怎样验证
文档入口与已知限制

维护者作判断

结合需求与整体设计
核对关键证据与风险
决定接收、修改或暂缓

例会讲解：问题背景 → 改动范围 → 验证结果 → 本次需要的决定；会后回写PR。

不是写不过Agent才转做review；有依据的验证和反馈，本身就是贡献。

帮忙 review 也是贡献，不必等到成为 Reviewer

不必另开一个PR，也能推进已有工作

1 复现 Issue，补足问题信息

在什么环境能复现？补步骤与日志；
已有同类讨论时，附链接说明关系。

2 验证 PR，检查修改前后

原问题是否解决？相关旧示例是否可用？
成功结果与未验证范围都值得反馈。

3 读相关代码与文档

指出具体位置、预期行为与依据；
从熟悉的模块看，不必一次审完整个PR。

一条有帮助的 comment

教学模板：方括号填写实际结果

我在[环境 / commit]按[命令]验证。
修改前：[现象]；修改后：[结果]。

相关旧示例[名称]：[结果]。
尚未验证：[范围与原因]。

日志或代码位置：[链接]。

review同样能借助Agent；真正有价值的是新增证据、具体判断和后续跟进。

不只增加代码量，逐步承担完整问题

从你熟悉的一小部分开始，用Agent补足实现与验证能力

恢复一个可用示例

复现问题 → 修复 → 旧示例回归 → 文档与review

不只在自己机器上跑通，让下一个使用者也能照做。

建立可复现的评测

数据与配置 → 基线 → 对照实验 → 结果边界

不只生成一张结果表，让社区能判断方案是否有用。

补上自动化检查

确认重复问题 → 编写检查 → 接入流程 → 验证误报

不只完成一次检查，让后续贡献更容易被验证。

让Agent承担更多实现，把自己的精力逐步放到需求、验收、取舍和持续推进上。

第一个 PR 可以简单，但不要把“刷数量”当成长

我们通常更愿意鼓励第一次贡献

真实的小bug修复可以是起点；愿意多给指导，不等于免除质量要求。

合理拆分：让审查更容易

每个PR目标明确、可以验证
拆分后保持可用，说明前后依赖
例：先做兼容性重构，再接新功能

刻意拆碎：只是为了增加计数

同一个修复按文件 / 几行拆成多单
单个PR没法独立说明或验证
让维护者反复补上下文、重复审查

小贡献仍可作为起点；每次实践都应增加自己的理解，而不只是机械完成任务。

用Agent补短板，在实践中积累判断力

借工具开始

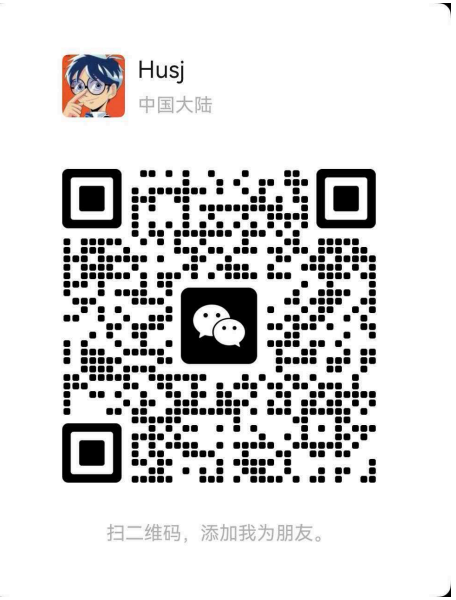
借助Agent跨过编码障碍
从一个真实小问题开始
不必先学会所有技术

在任务中判断

先预测，再验证与纠偏
说清选择、证据和限制
把经验留给下一次任务

把工作推进

贡献代码，也补可靠反馈
回应review、持续跟进
逐步承担更完整的工作



申请备注：开源软件课程

没有变：真实需求、工程质量、贡献责任、持续协作。

实现门槛降低，不等于贡献价值自动增加。
借工具补短板，在真实问题中积累自己的判断力。